

Introduction to Vector Search 2.0...

Language ▾

<https://acc>

1 Introduction (#0)

2 Building TheLook Fashion Search (#1)

3 Congratulations (#2)

requests, handle rate limits, and store the vectors. Every time your data changes, you re-run the pipeline. It's infrastructure you have to build before you can even start searching.

The feature store. Many vector search products provide only a vector index that returns a list of item IDs for each search. To serve full search results to users, you need a separate [feature store](https://cloud.google.com/vertex-ai/docs/featurestore/latest/overview) (<https://cloud.google.com/vertex-ai/docs/featurestore/latest/overview>) or [key-value store](https://cloud.google.com/bigtable/docs/overview) (<https://cloud.google.com/bigtable/docs/overview>) to retrieve the actual item data — names, prices, categories, image URLs in millisecs — by passing those IDs. In many cases, you also need to implement complex filtering on item features such as price, category, or availability. This means building and maintaining two different services: one for vector search, one for data retrieval and filtering. Every update and query requires accessing and syncing both systems.

The index tuning. To build [approximate nearest neighbor \(ANN\)](https://en.wikipedia.org/wiki/Nearest_neighbor_search) (https://en.wikipedia.org/wiki/Nearest_neighbor_search) indexes with millions of items, you need to make [expert decisions](https://cloud.google.com/vertex-ai/docs/vector-search/configuring-indexes) (<https://cloud.google.com/vertex-ai/docs/vector-search/configuring-indexes>) to get the best performance: How many items should each index node hold? What percentage of the index should be scanned per query to balance recall against latency? What shard size matches your dataset? These are ML infrastructure decisions that have nothing to do with your actual product.

The hybrid search. Semantic search excels at understanding intent — finding "Board Shorts" when users search "men's outfit for beach." But it fails on product codes like "SKU-12345" that have no semantic meaning, and struggles with newly coined terms or brand names the embedding model has never seen. Keyword search handles these cases but misses semantic context. Users need both, which is why [hybrid search](https://github.com/GoogleCloudPlatform/generative-ai/blob/main/embeddings/hybrid-search.ipynb) (<https://github.com/GoogleCloudPlatform/generative-ai/blob/main/embeddings/hybrid-search.ipynb>)

has become essential. Building it, however, is far from trivial. You need a full-text search engine with tokenization, inverted indexes, or sparse embeddings — in addition to your vector search engine. Then you must run parallel queries on both engines, normalize their different scoring systems, and merge results with techniques like Reciprocal Rank Fusion.

How Vector Search 2.0 Solves These Problems

Vector Search 2.0 on Google Cloud directly addresses each of these challenges:

